

Perubahan Algoritma Searching

Penjelasan perubahan algoritma pencarian pada *mini project* kami, yang seharusnya kami menggunakan algoritma *Binary Search* akan kami rubah menjadi algoritma *Linear Search*. Hal ini kami lakukan karena efisiensi dalam pencarian data.

Dimana pada *Binary Search* lebih efisien untuk pencarian satu data, sehingga tidak sesuai dengan *output* yang kami inginkan yaitu menampilkan semua kemungkinan yang ada. Contohnya Ketika kami ingin mencari data dengan *input* “ap”, *output* yang diharapkan adalah menampilkan semua data yang memiliki elemen “ap” pada nama data tersebut. Jika kami memaksakan menggunakan algoritma *Binary Search*, kami menemukan dua kemungkinan yang kami dapatkan dalam kode ini yaitu menggunakan *linear search* untuk semua nama produk dan menggunakan *Binary Search* kami mencari data harga dan deskripsi, seperti kode berikut ini:

```
# Linear Search Function
def linearSearch(arr,target):
    results = []
    for item in arr:
        if target.upper() in item['name']:
            results.append(item['name'])
    return results

def binarySearch (arr,target,low, high):
    if len(arr) >= 1:
        mid = (low+high)//2
        if target == arr[mid]['name']:
            return mid
        elif target < arr[mid]['name']:
            return binarySearch(arr,target,low,mid-1)
        else:
            return binarySearch(arr,target,mid+1, high)

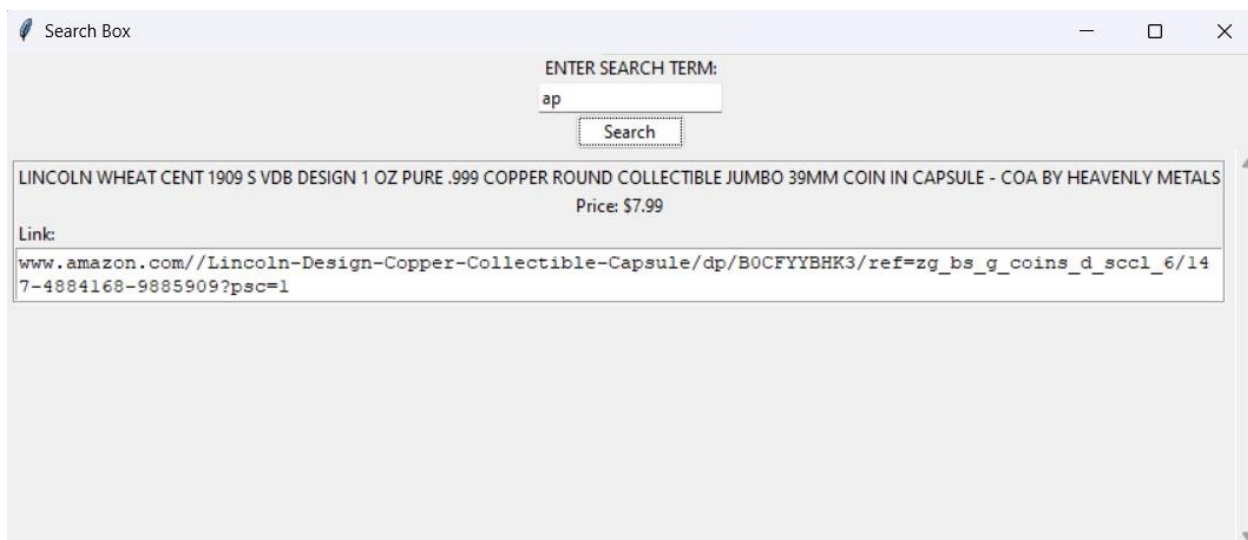
def search():
    search_term = entry.get()
    if search_term:
        clear_results()
        matches = linearSearch(sorted_arr, search_term)
        matches = []
        for product in match:
            low = 0
            high = len(sorted_arr)-1
            eachproduct = sorted_arr[binarySearch(sorted_arr, product, low, high)]
            matches.append(eachproduct)
        matches = matches[:100]
        if matches:
            for match in matches:
                display_result(match)
        else:
            display_result({"name": "No match found", "price": "", "description": ""})
```

walaupun *output* yang ditampilkan sesuai, tetapi kode ini tidak efisien karena data dari harga dan deskripsi bisa didapatkan dari *Linear Search*.

Sedangkan untuk kemungkinan kedua, kami menggunakan *Binary Search* untuk mencari semua kemungkinan, seperti pada kode berikut:

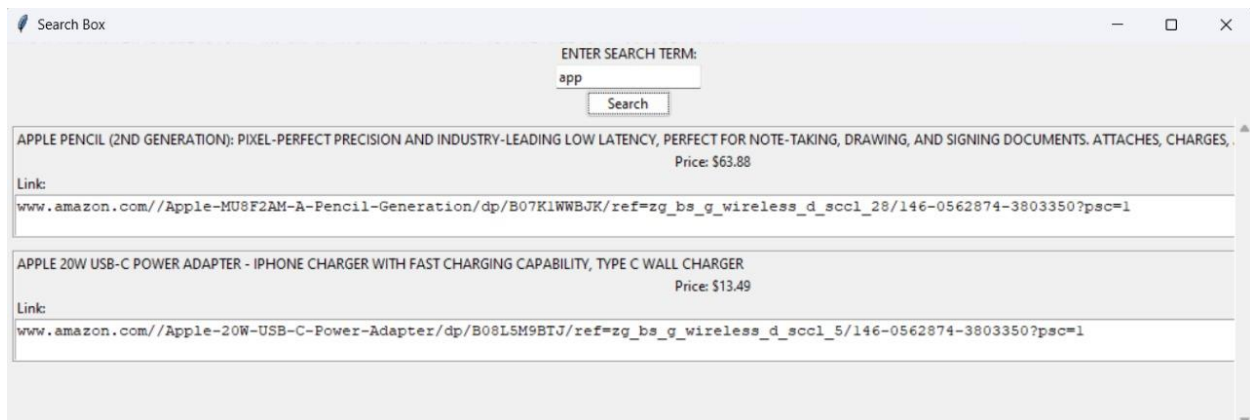
```
def binary_search(arr, x, low, high):
    result = []
    if low > high:
        return False
    elif low <= high:
        mid = (low + high) // 2
        if x.lower() in arr[mid]['name'].lower():
            result.append(arr[mid])
            left = mid - 1
            while left >= 0 and x.lower() in arr[left]['name'].lower():
                result.append(arr[left])
                left -= 1
            right = mid + 1
            while right < len(arr) and x.lower() in arr[right]['name'].lower():
                result.append(arr[right])
                right += 1
            return result
        elif arr[mid]['name'].lower() < x.lower():
            return binary_search(arr, x, mid + 1, high)
        else:
            return binary_search(arr, x, low, mid - 1)
```

Dimana semua hasil pencarian akan disimpan dalam list result yang akan dikembalikan dari fungsi tersebut. Namun hasil yang ditemukan menjadi absurd, seperti yang terlampir pada gambar berikut ini:

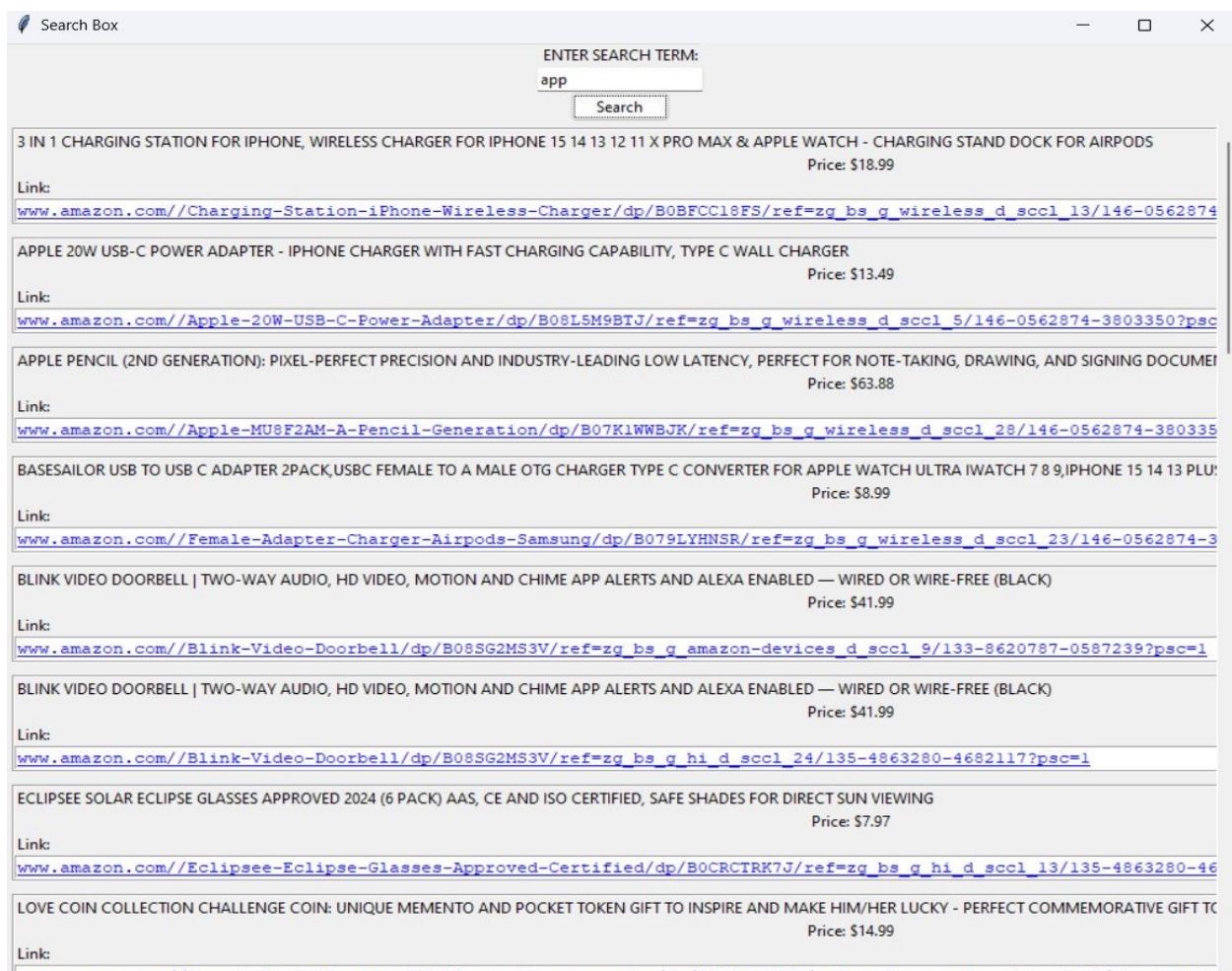


The screenshot shows a web browser window titled "Search Box". Inside, there is a search interface with the text "ENTER SEARCH TERM:" above a text input field containing "ap". Below the input field is a "Search" button. The search results are displayed in a box below the input field, showing a product listing for "LINCOLN WHEAT CENT 1909 S VDB DESIGN 1 OZ PURE .999 COPPER ROUND COLLECTIBLE JUMBO 39MM COIN IN CAPSULE - COA BY HEAVENLY METALS" with a price of \$7.99. Below the product listing is a "Link:" label followed by the Amazon URL: www.amazon.com/Lincoln-Design-Copper-Collectible-Capsule/dp/B0CFYVBHK3/ref=zg_bs_g_coins_d_sccl_6/147-4884168-9885909?psc=1.

Hal berikutnya ketika kami mencari data dengan kata kunci “ap”, kami hanya menemukan satu hasil. Sedangkan Ketika kami mencari data dengan kata kunci “app” hasil yang ditampilkan terdapat 2 data, seperti gambar dibawah ini:



Dimana dalam data yang kami miliki terdapat banyak data yang mengandung elemen “app”. Hal ini dapat dilihat ketika kami menggunakan algoritma *Linear Search* seperti gambar dibawah ini:



Dengan demikian karena pertimbangan ini kami menyimpulkan bahwa penggunaan algoritma *Linear Search* lebih efisien untuk mencari banyak data dan menampilkan semua data yang sesuai dengan kata kunci. Oleh karena itu kami merubah algoritma pencarian kami dari *Binary Search* menjadi *Linear Search*.